

Guatemala, 2/08/2026

Universidad del Valle de Guatemala, Ing. CCTI

Redes, sección 10

Melisa Dayana Mendizabal Meléndez 23778

Renato Manuel Rojas Roldán 23813

Lab2 - Esquemas de detección y corrección de errores

Descripción:

Esta práctica está enfocada a los métodos de detección y corrección de errores utilizados durante la transmisión de información. Se envía un mensaje desde un lenguaje de programación, en forma de socket, y el mismo se recibe desde otro programa ejecutándose (creado con otro lenguaje de programación). Al momento de salir del emisor, este tiene una probabilidad de generarle ruido a los bits del mensaje original; el receptor debe de detectar o corregir el error (si es que es posible) mediante algoritmos bien conocidos del estándar de redes; finalmente, debe mostrar si el mensaje fue corregido con éxito (o no presentó ninguna alteración) y mostrar el mensaje, o por el contrario, que no se pudo corregir el error de ruido en el mensaje.

Resultados:

El lenguaje de programación desde donde fue enviado el mensaje fue Python, mientras que el lenguaje que recibe se decidió fuera C. Aquí se adjuntan capturas de pantalla de los resultados obtenidos durante la ejecución del código:

Resultados de enviar un mensaje con el método de corrección de VITERBI:

```
PS C:\Users\darta\OneDrive\Desktop\REN\UVG\2026\Segundo ciclo\Redes\lab2_redes\emisor> python main_emisor.py "buenas noches" VITERBI 0.05
Enviado: 212 bits (104 bits de carga + redundancia)
PS C:\Users\darta\OneDrive\Desktop\REN\UVG\2026\Segundo ciclo\Redes\lab2_redes\emisor> |

ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/Redes/lab2_redes/receptor:$ ./receptor 6
000
Escuchando en el puerto 6000...
(se detectaron y corrigieron errores de transmision)
Mensaje recibido: buenas noches
ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/Redes/lab2_redes/receptor:$
```

Resultado de ejecutar con el método de detección FLETCHER:

```

PS C:\Users\darta\OneDrive\Desktop\REN\UVG\2026\Segundo ciclo\Redes\lab2_redes\emisor> python main_emisor.py "hola" FLET
CHER 0.01
Enviado: 64 bits (32 bits de carga + redundancia)
PS C:\Users\darta\OneDrive\Desktop\REN\UVG\2026\Segundo ciclo\Redes\lab2_redes\emisor> python main_emisor.py "hola" FLET
CHER 0.01
Enviado: 64 bits (32 bits de carga + redundancia)
PS C:\Users\darta\OneDrive\Desktop\REN\UVG\2026\Segundo ciclo\Redes\lab2_redes\emisor> |

ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/Redes/lab2_redes/receptor$ ./receptor 6
000
Escuchando en el puerto 6000...
ERROR: se detectaron errores y el algoritmo no puede corregirlos.
ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/Redes/lab2_redes/receptor$ ./receptor 6
000
Escuchando en el puerto 6000...
Mensaje recibido: hola
ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/Redes/lab2_redes/receptor$ █

```

Ejecución de incorregible con VITERBI:

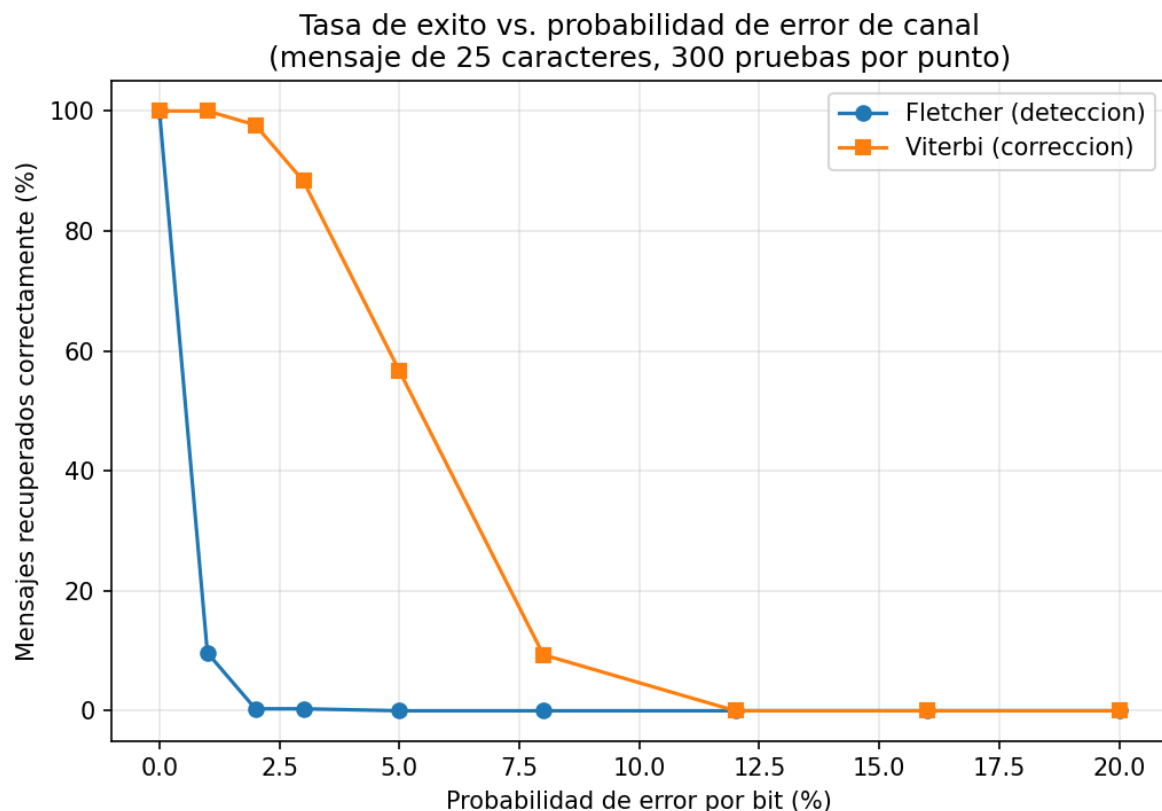
```

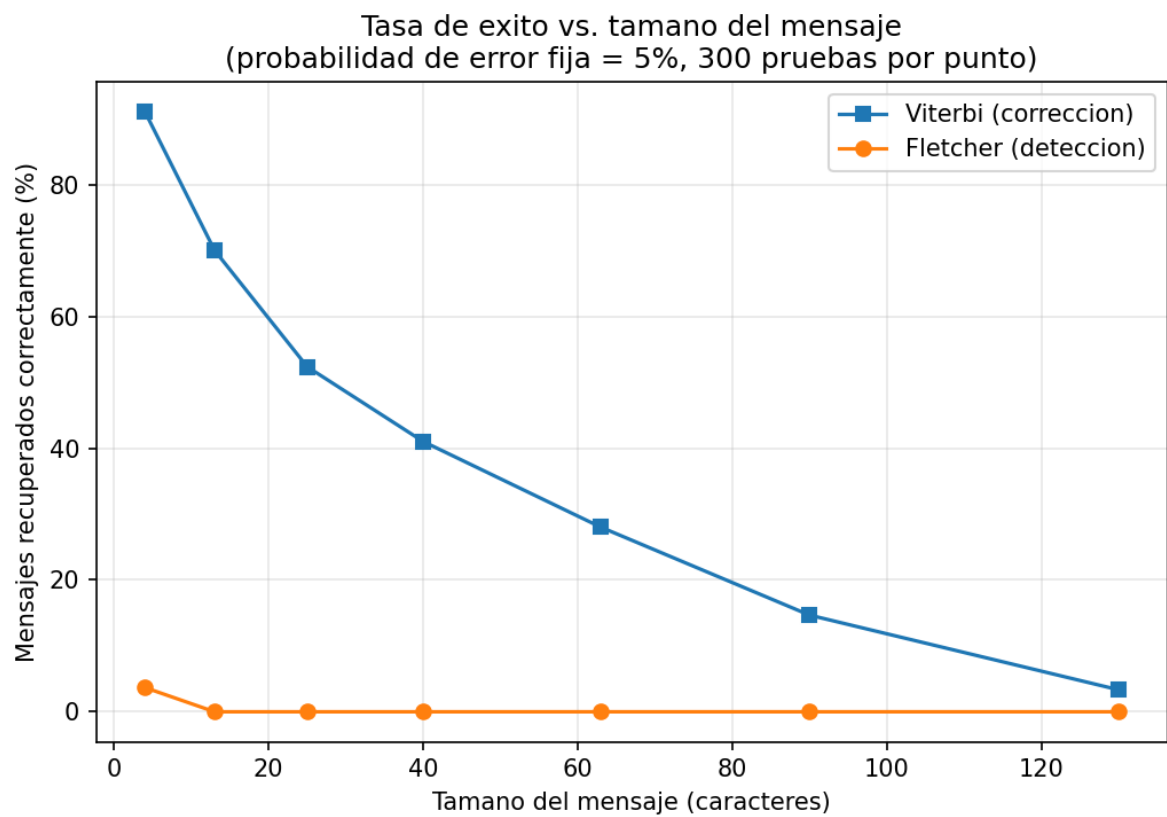
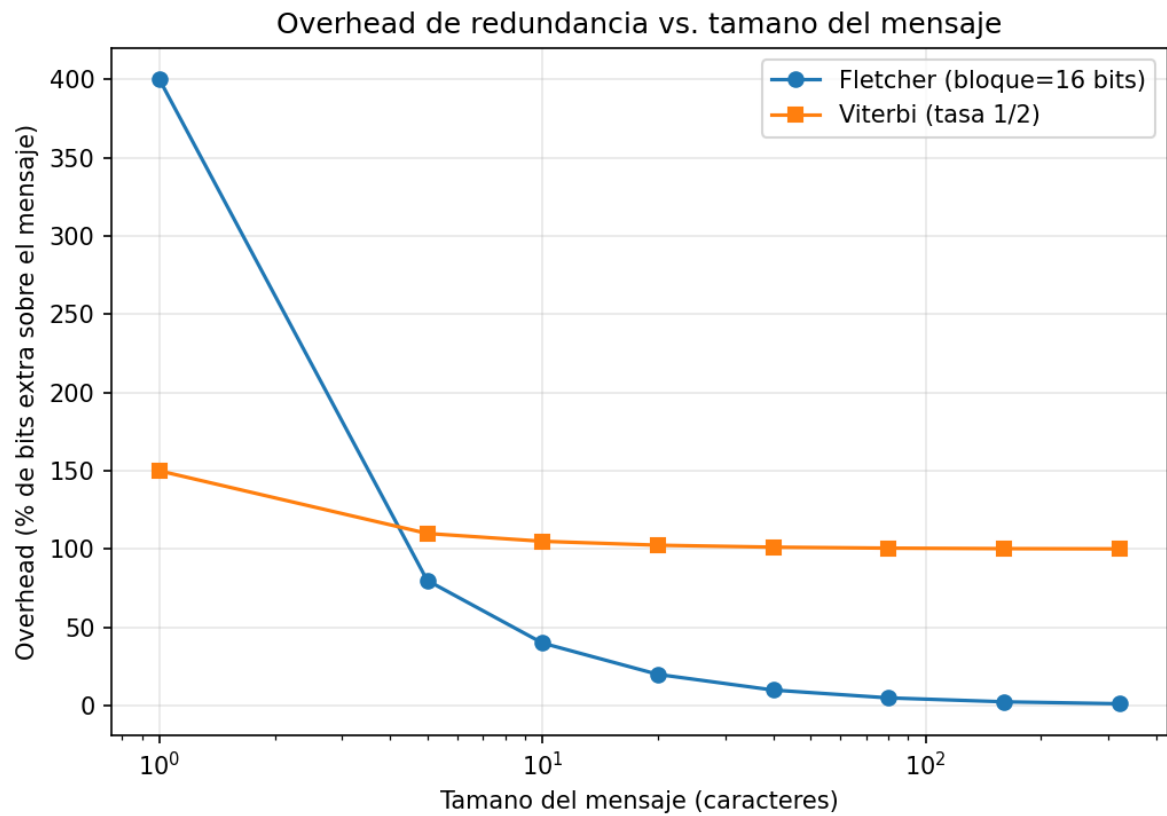
PS C:\Users\darta\OneDrive\Desktop\REN\UVG\2026\Segundo ciclo\Redes\lab2_redes\emisor> python main_emisor.py "este es un
mensaje considerablemente largo que costara corregir" VITERBI 0.05
Enviado: 1012 bits (504 bits de carga + redundancia)
PS C:\Users\darta\OneDrive\Desktop\REN\UVG\2026\Segundo ciclo\Redes\lab2_redes\emisor> |

ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/Redes/lab2_redes/receptor$ ./receptor 6
000
Escuchando en el puerto 6000...
(se detectaron y corrigieron errores de transmision)
Mensaje recibido: este es un mensaje considerablemente largo que costd"a corregc◊
ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/Redes/lab2_redes/receptor$ █

```

Resultados en gráficas de las pruebas automáticas:





Discusión:

Se aprendió que existen algoritmos que únicamente detectan problemas de transmisión, así como otros pueden no solo detectar, sino también intentar corregir los bits enviados; no obstante, incluso dichos algoritmos tienen sus límites. Algo interesante que se descubrió es que, si el mensaje original tiene menos caracteres, hay menos bits con probabilidad de ser afectados por el ruido. Por tanto, en base a esto, se puede determinar cuánto ruido puede soportar una cadena (además del parámetro de probabilidad de ruido que se le agrega al main de python, al momento del envío del mensaje). Esto se puede evidenciar en la tercera gráfica, pues entre mayor es el tamaño del mensaje, menores son los mensajes recuperados (con una clara caída casi inmediata por parte de Fletcher, pues no corrige, solo detecta).

A mayor detalle, lo de los límites de corrección se puede apreciar con el tercer ejemplo dentro de “Resultados”, en donde una cadena muy larga, a pesar de solo tener un 5% de probabilidad de ruido, puede llegar a tener una interferencia notoria, que puede cambiar el significado inicial del mensaje. En este caso, aunque claramente se entiende el mensaje, ya hay caracteres fallidos de corrección que no pudieron interpretarse correctamente como ASCII, mientras que posiblemente dentro de la misma cadena hubo otros caracteres que sí cambiaron, pero que también lograron corregirse gracias a la implementación del algoritmo VITERBI.

Se evidencia que VITERBI posee una mejor tasa de desempeño, pues no solo rechaza la cadena alterada como FLETCHER (que se diseñó de esa manera, simulando una implementación real), sino que intenta corregirlo lo más posible; no obstante, como punto negativo de esto mismo, puede dejar pasar un mensaje no corregido del todo (como el ejemplo 3 recién mencionado), lo que puede ocasionar un fallo si dos computadoras se están hablando entre sí (leerá un 1 en vez de un 0, lo que crearía un fallo). Por tanto, si el mensaje debe de ser interpretado por un ser humano y no por una computadora, entonces el uso de VITERBI (o cualquier otro algoritmo de corrección) convendrá más, pues algún mal bit o carácter faltante podrá ser interpretado por el operador; mientras que si se pretende que la comunicación sea totalmente entre máquinas, entonces es mejor implementar únicamente el de detección, para así rechazar los mensajes erróneos, y no ocasionar un error más grande a consecuencia de una mala interpretación de bits.

Para respaldar estas observaciones de forma cuantitativa, se realizaron pruebas automatizadas variando la probabilidad de error entre 0% y 20%, con 300 repeticiones por cada punto para reducir el efecto del azar propio del ruido aleatorio. Los resultados confirman lo observado de manera empírica en las capturas anteriores: FLETCHER pierde su capacidad de entregar el mensaje casi de inmediato (con apenas 1% de probabilidad de error, solo el 12.3% de los mensajes llegaron intactos, y a partir de 3% la tasa de éxito cae a 0%), pues basta con que un solo bit se altere para que el algoritmo rechace la trama completa. VITERBI, en cambio, mantiene una tasa de éxito superior al

85% incluso con un 3% de ruido, y solo empieza a degradarse notoriamente entre el 5% y el 8%, punto en el que la cantidad de bits alterados supera la capacidad de corrección del algoritmo (esto explica precisamente el comportamiento del tercer ejemplo de la sección de Resultados, donde una cadena larga con 5% de ruido ya presentó caracteres que no pudieron corregirse del todo). A partir de 12% de probabilidad de error, ambos algoritmos convergen a una tasa de éxito de 0%, lo que evidencia que ningún esquema de corrección es infalible ante niveles de ruido suficientemente altos.

Esta ventaja de VITERBI, sin embargo, no es gratuita. Al analizar el overhead (bits de redundancia añadidos) en función del tamaño del mensaje, se observa que FLETCHER agrega una cantidad fija de bits de verificación (el doble del tamaño de bloque, en este caso 32 bits), por lo que su overhead relativo disminuye considerablemente entre más largo sea el mensaje (de 400% para un mensaje de un solo carácter, hasta menos del 5% para mensajes de 80 caracteres o más). VITERBI, al ser un código de tasa 1:2, mantiene un overhead prácticamente constante de aproximadamente 100% sin importar la longitud del mensaje, ya que cada bit de entrada siempre genera dos bits de salida. Esto confirma lo mencionado en las conclusiones sobre el gasto computacional de cada algoritmo: la capacidad de corrección de VITERBI tiene como costo duplicar permanentemente el ancho de banda utilizado, mientras que FLETCHER es más eficiente en mensajes largos, pero a cambio de no ofrecer ninguna tolerancia a errores más allá de su detección.

Conclusiones:

Se comprendió el funcionamiento de un modelo de comunicación de capas, evidenciando cómo la información debe de atravesar todas ellas (de salida y entrada) para poder ser transmitida correctamente.

Existen varios algoritmos de detección y de detección-corrección de mensajes transmitidos durante la transmisión de mensajes (bits). Unos implican un mayor gasto computacional, y también pueden estar más optimizados para ciertos lenguajes que para otros.

Depende del tipo de sistema que se esté utilizando o creando, es más conveniente utilizar un algoritmo de detección o de corrección; depende de los contextos y objetivo final del sistema.

Referencias:

University of New Brunswick. (s.f.). Convolutional Coding - Viterbi Decoding.
<https://www.ece.unb.ca/tervo/ece4253/convolution3.shtml>

Wikipedia. (2026). Algoritmo de Viterbi. https://en.wikipedia.org/wiki/Viterbi_algorithm

Yadav, C. (2026). Fletcher's checksum. <https://www.tutorialspoint.com/article/fletcher-s-checksum>

Anexos:

Link al repositorio: https://github.com/Ren23813/lab2_redes